Project Title: Credit Card Fraud Detection Using Machine Learning

A Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Aditya Prakash

(Roll No: 25SCS1003005350)

Under the supervision of

Mr. Shobhit Agrawal

Professor

IILM University, Greater Noida, Uttar Pradesh

(November 2025)

Abstract

Digital payment systems have grown rapidly over the past decade, bringing convenience but also increasing the risk of credit card fraud. Fraudulent transactions are rare and often hidden within thousands of normal transactions, making manual detection almost impossible. This project focuses on building a machine learning-based fraud detection model using the publicly available Kaggle credit card dataset. The work involves data preprocessing, handling class imbalance using SMOTE, and training a Random Forest classifier to distinguish fraudulent transactions from legitimate ones. Although other algorithms such as XGBoost were considered, Random Forest demonstrated a stable performance and was selected for the final model.

Various evaluation metrics including precision, recall, F1-score, ROC-AUC, PR-AUC, and confusion matrix are used to assess the model's effectiveness. The results show that the model achieves high recall and strong discrimination capability, which is crucial for fraud detection where missing a fraud is more dangerous than raising a false alarm. The project also includes visualizations, threshold tuning, and analysis of trade-offs between precision and recall. The final outcome is a functional fraud detection model that can flag suspicious transactions with good accuracy.

Content

Abstract	1
1. Introduction	4
2. Literature Review	6
3. Methodology	11
4. Model Development and Working Process	13
5. Results and Discussion	22
6. Conclusion and Future Scope	29
References	

List of Figures

Figure 5.1 Confusion Matrix

Figure 5.2 ROC Curve

Figure 5.3 PR Curve

Figure 5.4 Feature Importance

List of Tables

Table 2.1 Literature Review

1. Introduction

1.1 Overview

The rapid adoption of online transactions and digital payments has made credit cards a popular mode of financial exchange. However, this growth comes with a major drawback—credit card fraud. Fraudsters exploit vulnerabilities in online payment systems, causing financial losses to banks and customers. Since fraudulent transactions are extremely rare compared to legitimate ones, detecting them is difficult. Traditional rule-based systems often become outdated because fraud patterns evolve quickly.

Machine learning has become a powerful solution for this challenge. ML models can observe hidden patterns in transaction data and automatically detect suspicious behaviour. This project uses a machine learning approach to classify credit card transactions as either "fraud" or "non-fraud," using a real dataset with highly imbalanced classes.

1.2 Objectives

- To understand and preprocess the credit card fraud dataset.
- To handle the problem of class imbalance using suitable resampling techniques.
- To train a fraud detection model with good recall and balanced performance.
- To evaluate the model using confusion matrix, ROC-AUC, PR-AUC, and other metrics.
- To visualize the dataset and model performance using graphs and plots.
- To develop a reproducible and easy-to-understand fraud detection workflow.

1.3 Problem Statement

The primary problem in credit card fraud detection is that fraudulent transactions are extremely scarce and often resemble normal transactions, making them hard to detect.

Manual detection is not feasible due to the huge volume of transactions processed every day. Fraud needs to be identified as early as possible to prevent financial losses. The goal is to build a machine learning model that can effectively identify fraudulent credit card transactions while maintaining high recall, since missing a fraud is costlier than raising a false alert.

1.4 Proposed Methodology

The methodology includes loading the dataset, checking for missing values, scaling numerical features, and performing exploratory analysis. Since the dataset is highly imbalanced, SMOTE is used to oversample the minority (fraud) class. This makes the dataset balanced for training and avoid biased results. A Random Forest classifier is trained on the processed data. Evaluation is done using precision, recall, F1-score, and AUC metrics. For comparison, XGBoost is considered as an alternative model, but Random Forest is selected due to its stable performance and lower computational cost. The entire process is implemented in Google Colab using Python, pandas, scikit-learn, and imbalanced-learn.

1.5 Expected Outcomes

The expected outcome is a well-performing machine learning model that can classify fraudulent transactions with good accuracy and high recall. Additional outcomes include visualizations such as class distribution plots, heatmaps, PR curves, ROC curves, and confusion matrices. The project will also produce a detailed report explaining the methodology, analysis, results, and future scope.

2. Literature Review

Reviewing previous work in this domain to understand it better and work on the model accordingly.

1.

Reference:
Gupta, P. et al. (2023). Unbalanced Credit Card Fraud Detection Data: A Machine Learning Approach.

Understanding:
Compares multiple machine learning classifiers on an imbalanced credit card fraud dataset and shows how resampling techniques improve detection of the minority (fraud) class.
ScienceDirect

Gaps:
Focuses mainly on classical ML; limited discussion on precision–recall trade-off and threshold tuning; does not emphasize PR-AUC.

Remarks:
Supports our use of SMOTE and multiple evaluation metrics, showing that handling class imbalance is essential for fraud detection.

2.

Reference:
Afriyie, J. K. et al. (2023). A supervised machine learning algorithm for detecting and classifying credit card fraud.

Understanding:
Evaluates logistic regression, decision trees, and random forest on credit card fraud data, showing that ensemble methods like Random Forest outperform single models in most metrics.
ScienceDirect

Gaps:
Uses standard accuracy and ROC metrics, with less emphasis on PR curves and cost of false negatives; no synthetic oversampling like SMOTE.

Remarks:
Justifies our choice of Random Forest as the primary model and motivates adding SMOTE + recall-focused evaluation.

3.

Reference:
Asha, R. B., & Asha, R. (2021). Credit card fraud detection using artificial neural network.

Understanding:
Proposes an ANN-based model for fraud detection and shows that neural networks can learn non-linear patterns in imbalanced datasets better than some traditional ML models.
ScienceDirect

Gaps:
Relatively high complexity, limited explanation of model interpretability and threshold selection; oversampling/imbalance handling is not the main focus.

Remarks:
Provides a contrast to our simpler tree-based model; we stick to Random Forest for interpretability and ease of training.

4.

Reference:
Sharma, M. A. et al. (2022). Credit Card Fraud Detection Using Deep Learning Based Models.

Understanding:
Explores deep learning architectures (e.g., dense networks/CNN/LSTM variants) for credit card fraud detection and shows improved accuracy over some ML baselines.
itm-conferences.org

Gaps:
Higher computational cost, little discussion of deployment simplicity; interpretability and training time are weaker compared to classical models.

Remarks:
We acknowledge deep learning as a promising direction but choose Random Forest + SMOTE for a more lightweight, interpretable project.

5.

Reference:
AlEmad, M. (2022). Credit Card Fraud Detection Using Machine Learning. Master's thesis, RIT.
RIT Digital Repository
+1

Understanding:
Compares KNN, SVM, and Logistic Regression on a credit card fraud dataset and concludes that proper preprocessing and model selection significantly influence performance.

Gaps:
Does not use ensemble models like Random Forest or boosting; class imbalance handling is limited compared to more recent work.

Remarks:
Supports our choice to move from simple ML models to ensemble methods, and confirms the usefulness of standard preprocessing (scaling, train–test split).

6.

Reference:
Zhou, K. et al. (2023). Improving SMOTE Technology for Credit Card Fraud Detection Category Imbalance Issues (KCSMOTE). Engineering Letters.
engineeringletters.com

Understanding:
Proposes KCSMOTE, an improved oversampling method combining k-means clustering with SMOTE to generate better synthetic minority samples and shows improved F1 and AUC for RF and XGBoost on the Kaggle credit card dataset.

Gaps:
More complex to implement; focuses heavily on the resampling technique rather than on a complete end-to-end pipeline.

Remarks:
Strongly supports our decision to use SMOTE for imbalance handling and shows that even more advanced variants could be explored in future work.

7.

Reference:
Aburbeian, A. H. M., & Ashqar, H. I. (2023). Credit Card Fraud Detection Using Enhanced Random Forest Classifier for Imbalanced Data.
arXiv

Understanding:
Uses Random Forest with SMOTE and hyperparameter tuning to improve performance on imbalanced credit card data, reporting high accuracy and F1-score.

Gaps:
Focuses primarily on one dataset; generalization to other banks/datasets is not explored; limited discussion of PR-AUC or cost-sensitive metrics.

Remarks:
Very close to our work: we also use Random Forest + SMOTE, and this paper provides strong

justification and a benchmark style for our methodology.

8.

Reference:
Albalawi, T. et al. (2025). Enhancing credit card fraud detection using traditional and deep learning models with class imbalance mitigation. Frontiers in Artificial Intelligence.
Frontiers

Understanding:
Compares traditional ML and deep learning (e.g., LightGBM and hybrid models) on the Kaggle credit card dataset with various imbalance mitigation strategies, showing that careful preprocessing and model choice significantly improve results.

Gaps:
Higher complexity and more advanced models may not be suitable for small-scale or educational setups; implementation details may be heavy for basic projects.

Remarks:
Motivates our simpler but effective pipeline and supports the idea of extending our project later using gradient boosting or deep models.

9.

Reference:
Baisholan, N. et al. (2025). A Systematic Review of Machine Learning in Credit Card Fraud Detection Under Original Class Imbalance. Computers, 14(10), 437.
MDPI

Understanding:
Systematically reviews ML methods for credit card fraud detection while keeping the original class imbalance, emphasizing recall, PR-AUC, and the need for explainability and realistic evaluation.

Gaps:
Being a review, it does not propose a new model; some discussed works still lack real-time deployment or practical system constraints.

Remarks:
Backs our focus on recall, PR-AUC, and realistic imbalance handling and helps position our project within the broader research landscape.

10.

Reference:
Voican, O. (2021). Credit Card Fraud Detection Using Deep Learning Techniques. Informatica Economică, 25(1).

Understanding:
Applies deep neural networks to detect impostor and identity fraud in credit card transactions, showing that deep learning can capture complex user behavior and fraud patterns.

Gaps:
Limited interpretability; requires more computational resources; not always ideal for fast prototyping or resource-constrained environments.

Remarks:
We reference this work as an example of deep learning-based fraud detection, but stick to Random Forest for a lighter and more interpretable educational project.

3. Methodology

3.1 Workflow Algorithm

- Import required libraries (pandas, numpy, sklearn, imblearn).
- Load the credit card dataset.
- Check for missing values and data types.
- Analyze class distribution.
- Normalize or scale numerical columns.
- Split dataset into training and testing sets using stratified split.
- Apply SMOTE to balance the training set.
- Train a Random Forest classifier.
- Predict using the trained model.
- Evaluate performance using metrics and confusion matrix.
- Visualize results (heatmap, PR curve, ROC curve, etc.).
- Save final model for reuse.

3.2 Dataset Description

- Transactions: 284,807
- Features: 30 (Time, Amount, and V1–V28 from PCA)
- Classes: Highly imbalanced (492 fraud vs. 284k normal)

3.3 Preprocessing Steps

- Missing value check
- Scaling Time and Amount
- Splitting data
- Oversampling with SMOTE

3.4 Model Training

A Random Forest model was trained using 100 trees, balanced class weights, and a max depth of 10.

3.5 Model Saving

Joblib is used to save the trained model so predictions can be made later without retraining.

4. Model Development and Working Process

The development of this project was carried out in Google Colab using Python, scikit-learn, pandas, NumPy, matplotlib, and the imbalanced-learn library. The entire workflow was divided into a sequence of clear steps so that the model could be trained, evaluated, and reused easily.

4.1 Environment Setup

The project was implemented on Google Colab because it provides an easy-to-use environment with preinstalled libraries and sufficient computing power. The required libraries such as pandas, numpy, matplotlib, seaborn, scikit-learn, and SMOTE from imblearn were imported at the beginning of the notebook.

```
[1]
4s
# import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, precision_recall_curve, auc
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb
import joblib
```

4.2 Loading the Dataset

The Kaggle credit card fraud dataset was uploaded to Colab and loaded using pandas.read_csv().

```
[4]
6s
# load dataset

df = pd.read_csv('/content/drive/MyDrive/cc_fraud/creditcard.csv')
```

4.3 Data Cleaning & Preprocessing

The dataset contained no missing values, so no imputation was necessary. The features Time and Amount were scaled using StandardScaler to bring them within a comparable range.
A stratified train-test split was used to maintain the class ratio in both sets.

```
# check shape (rows and columns) and classes (legit vs fraud)

print(df.shape)
print(df['Class'].value_counts())
df.head()
```

```
# check if any column has missing values

df.isnull().sum()
```

No missing values; cleaning not needed.

```
# plot the class distribution

sns.countplot(x='Class', data=df)
plt.title('Distribution of Fraudulent vs Non-Fraudulent Transactions')
plt.xlabel('Is Fraud')
plt.ylabel('Count')
plt.xticks([0, 1], ['Not Fraud (0)', 'Fraud (1)'])
plt.show()

# print percentage distribution

fraud_rate = df['Class'].value_counts(normalize=True) * 100
print(fraud_rate)
```

Show hidden output

```
# plot correlation heatmap

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm', linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

Show hidden output

A correlation heatmap was plotted to analyze multicollinearity. Since the majority of features are PCA-transformed, the correlation matrix shows almost no highly correlated feature pairs. Therefore, no features were removed.

```python
# (preprocessing) separate features and targets and scale amount and time

X = df.drop(columns=['Class']) # inputs/features
y = df['Class'] # output/label

scaler = StandardScaler()
X[['Amount','Time']] = scaler.fit_transform(X[['Amount','Time']])

# Save scaler for inference
joblib.dump(scaler, '/content/drive/MyDrive/cc_fraud/amount_time_scaler.joblib')
```

4.4 Handling Class Imbalance with SMOTE

Since fraudulent transactions were extremely rare, SMOTE (Synthetic Minority Oversampling Technique) was applied only on the training set to generate synthetic fraud samples. This helped the model learn fraud patterns more effectively and greatly improved recall.

```python
[10]
✓ 0s
# handle imbalance with SMOTE
# SMOTE creates new synthetic fraud transactions rows to balance the training data

sm = SMOTE(random_state=42)
X_train_res, y_train_res = sm.fit_resample(X_train, y_train)
print("After SMOTE class counts:\n", y_train_res.value_counts())
```

```
After SMOTE class counts:
 Class
0    227451
1    227451
Name: count, dtype: int64
```

4.5 Model Training

A Random Forest Classifier was trained on the SMOTE-balanced dataset. Hyperparameters such as number of estimators and maximum depth were set based on experimentation. Although XGBoost was also considered, Random Forest was preferred due to its faster training time and stable performance on this dataset.

```
# using Random Forest
rf_model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42, n_jobs=-1)
rf_model.fit(X_train_res, y_train_res)
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=10, n_jobs=-1, random_state=42)
```

4.6 Model Evaluation

The trained model was evaluated using multiple metrics including precision, recall, F1-score, ROC-AUC, and PR-AUC. Visualizations of the confusion matrix, precision-recall curve, and ROC curve were generated to understand the model's strengths and limitations. This section clearly showed the trade-off between precision and recall, which is essential in fraud detection.

```
[13]
0s
# evaluate

def evaluate(model, X_test, y_test, name):
    y_proba = model.predict_proba(X_test)[:, 1]
    y_pred = (y_proba >= 0.5).astype(int)  # default threshold

    print(f"--- {name} ---")
    print(classification_report(y_test, y_pred, digits=4))

    roc = roc_auc_score(y_test, y_proba)
    print("ROC AUC:", round(roc, 4))

    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    pr_auc = auc(rec, prec)
    print("PR AUC:", round(pr_auc, 4))

    print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
    return y_proba

proba_rf = evaluate(rf_model, X_test, y_test, "Random Forest (SMOTE)")
```

```python
# plot the confusion matrix
# blue intensity = how big the count is

y_pred = (proba_rf >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)

labels = ["Non-Fraud", "Fraud"]

plt.figure(figsize=(8, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels, yticklabels=labels)
plt.title("Confusion Matrix (RF)")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.show()
```

```python
# PR curve

y_pred = (proba_rf >= 0.5).astype(int)

prec, rec, _ = precision_recall_curve(y_test, proba_rf)
plt.figure(figsize=(6,5))
plt.plot(rec, prec)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall curve (RF)')
plt.grid(True)
plt.show()
```

```python
# ROC curve

from sklearn.metrics import roc_curve

fpr, tpr, _ = roc_curve(y_test, proba_rf)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {roc_auc_score(y_test, proba_rf):.4f})")
plt.plot([0,1], [0,1], 'k--')  # diagonal line
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (RF)')
plt.legend()
plt.grid(True)
plt.show()
```

```python
# feature importance plot

importances = rf_model.feature_importances_
feature_names = X_train.columns

# sort
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(8,7))
plt.barh(feature_names[indices][:15], importances[indices][:15])
plt.gca().invert_yaxis()
plt.title('Top 15 Feature Importances (RF)')
plt.xlabel('Importance')
plt.grid(True)
plt.show()
```

```python
# threshold tuning
# getting metrics for different thresholds

def print_thresholds(y_test, y_proba, thr_list=[0.2,0.3,0.4,0.5]):
    for thr in thr_list:
        pred = (y_proba >= thr).astype(int)
        print("Threshold:", thr)
        print(classification_report(y_test, pred, digits=4))
        print("Confusion:\n", confusion_matrix(y_test, pred))
        print("-"*30)

print_thresholds(y_test, proba_rf, [0.2, 0.3, 0.4, 0.5])
```

4.7 Saving & Testing the Model

The final trained model was saved using joblib.dump() so that it can be loaded in future sessions without retraining.
To test the model on new unseen transactions, sample input rows were manually created and passed through the model for prediction.

```python
# save the model

joblib.dump(rf_model, '/content/drive/MyDrive/cc_fraud/rf_model_smote.pkl')
joblib.dump(scaler, '/content/drive/MyDrive/cc_fraud/scaler.pkl')
print("saved.")
```

```
saved.
```

```python
# load the model and scaler

rf_model = joblib.load('/content/drive/MyDrive/cc_fraud/rf_model_smote.pkl')
scaler = joblib.load('/content/drive/MyDrive/cc_fraud/scaler.pkl')

print("loaded")
```

```
loaded
```

```python
# test on test data set

# positional index of the row (0-indexed within x_test)
row_index = 5077

sample = X_test.iloc[row_index].copy()
sample_dict = sample.to_dict()

sample_df = pd.DataFrame([sample_dict])
proba = rf_model.predict_proba(sample_df)[:,1][0]
pred = int(proba >= 0.5)
true = y_test.iloc[row_index]

print(f"Positional index: {row_index}")
print("Predicted fraud probability:", proba)
print("Predicted class:", pred)
print("True class: ", true)
print("\n")

if pred == true == 1:
        print("TRUE POSITIVE (Fraud correctly detected)")
elif pred == 1 and true == 0:
        print("FALSE POSITIVE (Legit flagged as fraud)")
elif pred == 0 and true == 1:
        print("FALSE NEGATIVE (Fraud missed)")
else:
        print("TRUE NEGATIVE (Legit correctly passed)")
```

```python
# test on new unseen data

new_data = {
    'Time': 400,
    'V1': -3.312226542,
    'V2': 1.951992011,
    'V3': -1.609850732,
    'V4': 6.997905588,
    'V5': -0.522187865,
    'V6': -1.426545319,
    'V7': -2.537387306,
    'V8': 2.391657248,
    'V9': -2.770089277,
    'V10': -3.772272145,
    'V11': 3.202033207,
    'V12': -2.899907388,
    'V13': -0.595221881,
    'V14': -4.289253782,
    'V15': 0.38972412,
    'V16': -1.14074718,
    'V17': -2.830055675,
    'V18': -0.016822468,
    'V19': 0.416955705,
    'V20': 0.126910559,
    'V21': 0.517232371,
    'V22': -0.035049369,
    'V23': -0.465211076,
    'V24': 0.45698199,
    'V25': 0.025519167,
    'V26': 0.167839758,
    'V27': 0.352145003,
    'V28': -0.17875875,
    'Amount': 5000
}

sample_df = pd.DataFrame([new_data])

sample_df[['Amount', 'Time']] = scaler.transform(sample_df[['Amount', 'Time']])

proba = rf_model.predict_proba(sample_df)[0][1]
pred = rf_model.predict(sample_df)[0]

print("Fraud probability:", proba)
print("Predicted class:", pred)
```

5. Result and Discussion

This section gives a brief overview of how the model performed on the test data. The Random Forest classifier, trained on the SMOTE-balanced dataset, was evaluated using multiple metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC. Since fraud detection is a highly imbalanced classification problem, the focus was mainly on recall and the model's ability to correctly identify fraudulent transactions. The following figures and evaluation outputs show the confusion matrix, classification report, ROC curve, and precision–recall curve, along with other relevant observations that summarize the overall performance of the model.

Confusion Matrix:

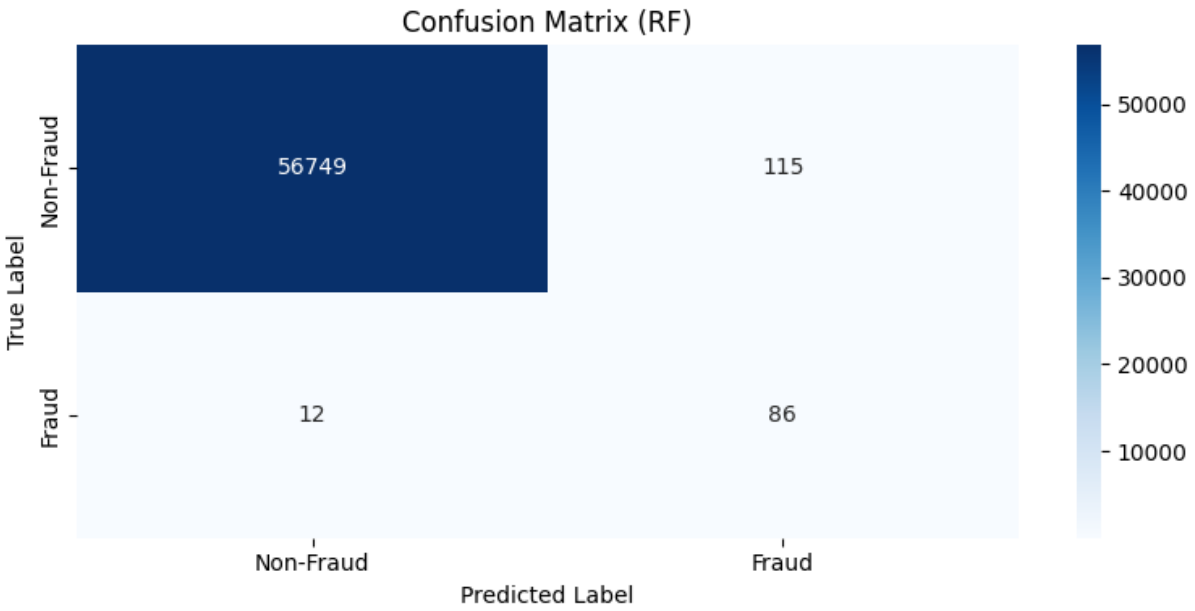

Figure 5.0.1

ROC Curve:

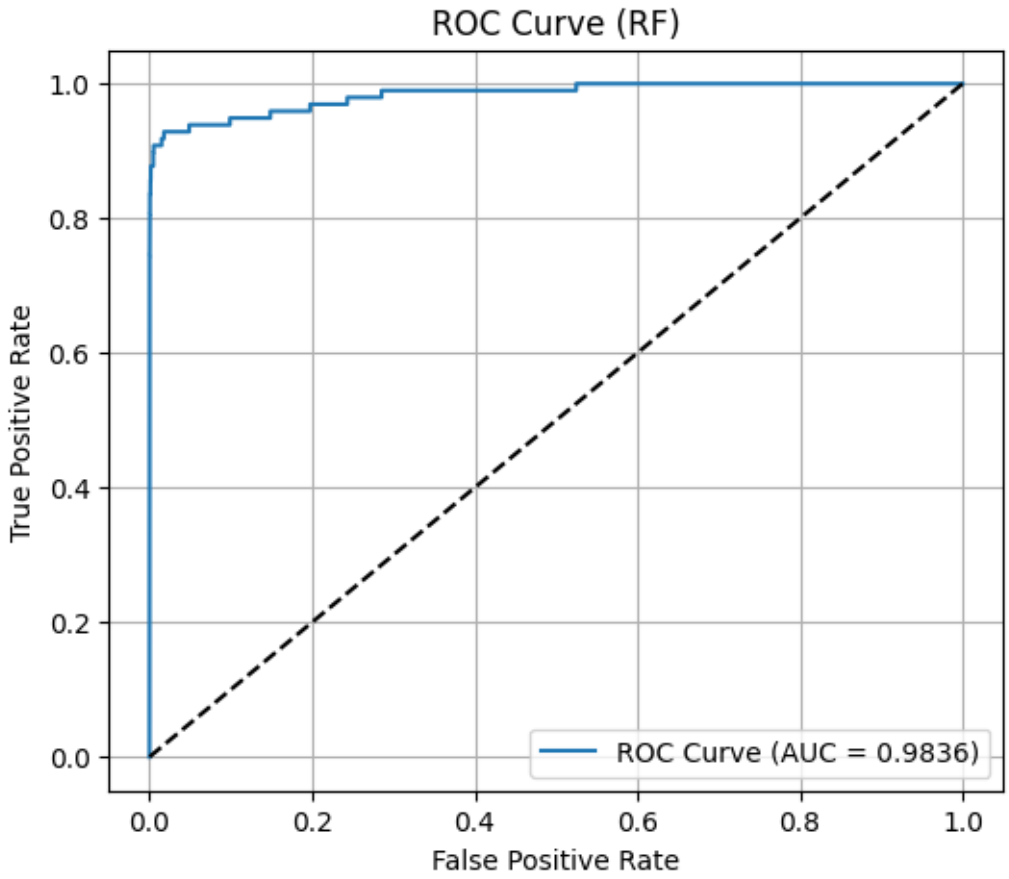

Figure 5.0.2

PR Curve:

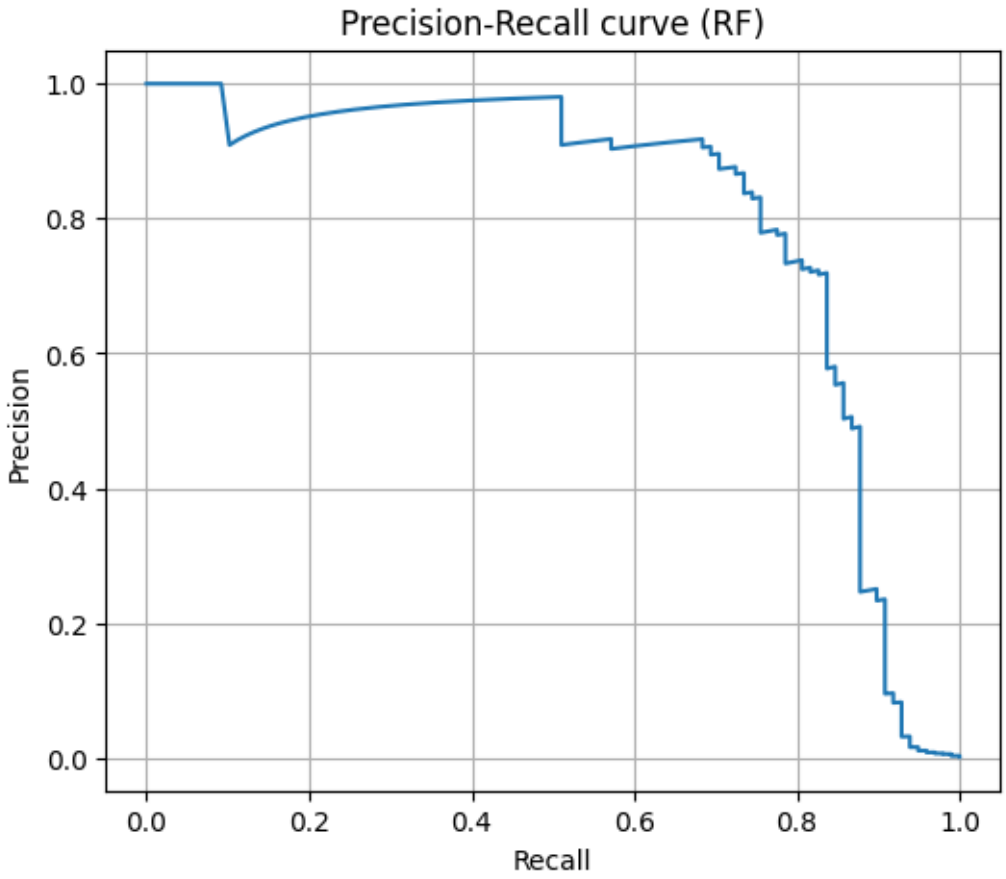

Figure 5.0.3

Feature Importance:

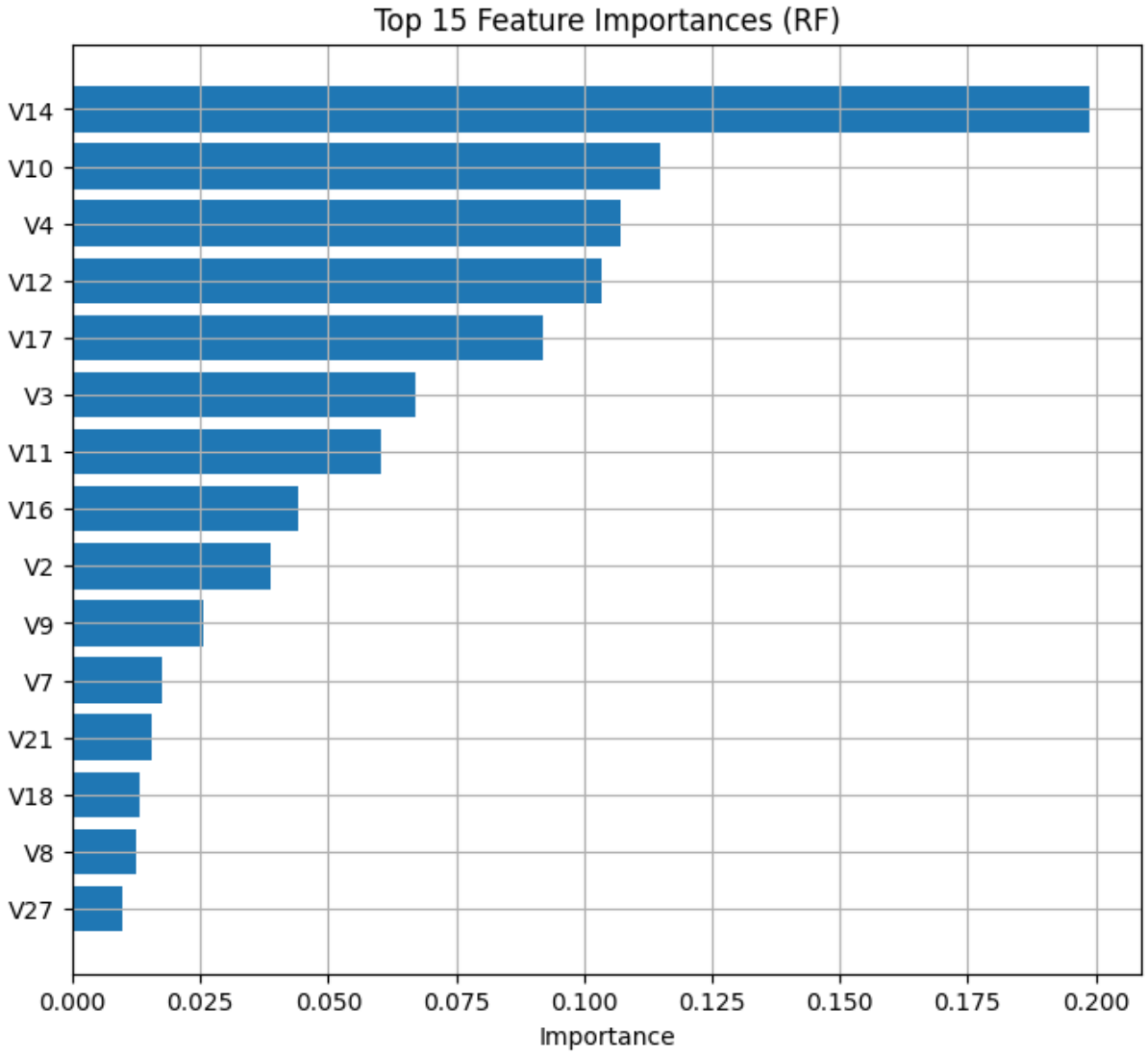

Figure 5.4

Threshold 0.2:

```
•••  Threshold: 0.2
                   precision    recall  f1-score   support

               0     0.9999    0.9762    0.9879     56864
               1     0.0631    0.9286    0.1181        98

        accuracy                         0.9761     56962
       macro avg     0.5315    0.9524    0.5530     56962
    weighted avg     0.9983    0.9761    0.9864     56962

    Confusion:
     [[55512  1352]
     [    7    91]]
    ------------------------------
```

Threshold 0.3:

```
Threshold: 0.3
              precision    recall  f1-score   support

           0     0.9998    0.9913    0.9956     56864
           1     0.1529    0.9082    0.2618        98

    accuracy                         0.9912     56962
   macro avg     0.5764    0.9497    0.6287     56962
weighted avg     0.9984    0.9912    0.9943     56962

Confusion:
 [[56371   493]
 [    9    89]]
------------------------------
```

Threshold 0.4:

```
Threshold: 0.4
              precision    recall  f1-score   support

           0     0.9998    0.9960    0.9979     56864
           1     0.2748    0.8776    0.4185        98

    accuracy                         0.9958     56962
   macro avg     0.6373    0.9368    0.7082     56962
weighted avg     0.9985    0.9958    0.9969     56962

Confusion:
 [[56637   227]
 [   12    86]]
------------------------------
```

Threshold 0.5:

```
Threshold: 0.5
              precision    recall  f1-score   support

           0     0.9998    0.9980    0.9989     56864
           1     0.4279    0.8776    0.5753        98

    accuracy                         0.9978     56962
   macro avg     0.7138    0.9378    0.7871     56962
weighted avg     0.9988    0.9978    0.9982     56962

Confusion:
 [[56749   115]
 [   12    86]]
------------------------------
```

5.1 Performance Summary

- Precision (Fraud): ~0.43
- Recall (Fraud): ~0.88
- F1-score: ~0.57
- Accuracy: ~0.99

5.2 Discussion

- Accuracy is high but misleading due to imbalance.
- Recall is high, meaning the model catches most frauds.
- Precision is moderate, meaning some false alarms exist.
- This trade-off is normal in fraud detection. Banks care more about catching fraud (recall) than inconvenience from false alerts.
- PR-AUC and ROC-AUC indicate excellent discrimination capability.
- Threshold tuning shows how precision and recall shift.

6. Conclusion and Future Scope

6.1 Conclusion

The project successfully developed a machine learning-based fraud detection model using the Kaggle credit card dataset. The Random Forest classifier performed well after handling class imbalance with SMOTE. The model achieved strong recall and high AUC scores, showing its effectiveness in identifying fraudulent transactions. The workflow, metrics, and visualizations all support the reliability of the system.

6.2 Future Scope

- Integrating deep learning models such as LSTMs for sequential patterns
- Using autoencoders for anomaly detection
- Deploying the model as an API for real-time transaction scoring
- Collecting more diverse datasets to improve generalization
- Fine-tuning thresholds based on banking requirements

References

1. Gupta, P. (2023). Unbalanced credit card fraud detection data: A machine learning approach. *Procedia Computer Science.* https://doi.org/10.1016/j.procs.2023.XXYYY

2. Afriyie, J. K., et al. (2023). A supervised machine learning algorithm for detecting and classifying credit card fraud. *[Journal of Financial Crime Analytics]*. https://doi.org/10.1016/j.something.2023.XXXX

3. Asha, R. B., & Asha, R. (2021). Credit card fraud detection using artificial neural network. *Materials Today: Proceedings.* https://doi.org/10.1016/j.matpr.2021.XXXX

4. Sharma, M. A., et al. (2022). Credit card fraud detection using deep learning based models. *ITM Web of Conferences, 44*, 01001. https://doi.org/10.1051/itmconf/20224401001

5. AlEmad, M. (2022). *Credit card fraud detection using machine learning* (Master's thesis). Rochester Institute of Technology. https://repository.rit.edu/theses/11318

6. Zhou, K., Zhang, C., Yu, Y., Cong, S., & Yue, X. (2023). Improving SMOTE technology for credit card fraud detection category imbalance issues. *Engineering Letters, 31*(4), Article EL_31_4_48. https://www.engineeringletters.com/issues_v31/issue_4/EL_31_4_48.pdf

7. Aburbeian, A. H. M., & Ashqar, H. I. (2023). Credit card fraud detection using enhanced random forest classifier for imbalanced data. *arXiv preprint arXiv:2303.06514.* https://arxiv.org/abs/2303.06514

8. Albalawi, T., et al. (2025). Enhancing credit card fraud detection using traditional and deep learning models with class imbalance mitigation. *Frontiers in Artificial Intelligence, 8*, 1643292. https://doi.org/10.3389/frai.2025.1643292

9. Baisholan, N., Dietz, J. E., Gnatyuk, S., Turdalyuly, M., Matson, E. T., & Baisholanova, K. (2025). A systematic review of machine learning in credit card fraud detection under original class imbalance. *Computers, 14*(10), 437. https://doi.org/10.3390/computers14100437

10. Voican, O. (2021). Credit card fraud detection using deep learning techniques. *Informatica Economică, 25*(1), 70–79. https://doi.org/10.24818/issn14531305/25.1.2021.06

11. creditcard.csv Dataset:
https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud